Python Best Practices and PEP 8

1. PEP 8 Naming Conventions

Adhering to PEP 8 ensures consistency across Python codebases. Naming conventions are categorized as follows:

• `snake_case`: Used for functions, variables, methods, and module names.

• `CapWords (PascalCase)`: Used for class names and exception types.

• `UPPER_SNAKE_CASE`: Used for module-level global constants.

• `_leading_underscore`: Used for internal/private attributes or functions (`from module import *` ignores these).

• `__double_leading_underscore`: Triggers class attribute name mangling in inheritance hierarchies.

2. Import Organization and Guidelines

Imports should be grouped in a specific, standardized order with blank lines separating each group:

1. Standard library imports.
2. Third-party library imports.
3. Local application/library specific imports.

Imports should always be placed at the top of the file (after module docstrings) rather than inside functions, avoiding circular import issues and improving readability.

```
import os import sys import requests import pydantic from myapp.models import User
```

3. Error Handling Philosophy

Robust error handling involves catching specific exceptions rather than using bare `except:` clauses, which can swallow critical system interrupts or programming errors.

Python developers generally practice EAFP ('Easier to Ask for Forgiveness than Permission') over LBYL ('Look Before You Leap'), relying on `try-except` blocks. When logging caught exceptions, always use `logging.exception()` to capture complete traceback details.

```
import logging try: value = int(user_input) except ValueError as e:
logging.exception('Invalid integer provided: %s', e)
```

4. Performance Optimization Tips

Optimizing Python code requires understanding interpreter mechanics. Key performance guidelines include:

• Use list and generator comprehensions instead of manual `for` loops with `append()`.

• Use `str.join()` rather than the + operator for concatenating large numbers of strings.

• Define `__slots__` in classes with heavy instance creation to drastically reduce memory overhead.

• Profile code performance using `cProfile` or `py-spy` before applying micro-optimizations.

5. Modern Dependency Management

Managing virtual environments and dependencies has evolved beyond traditional `requirements.txt` files. Modern Python tooling embraces standardized PEP 518/621 build configurations and robust lockfile managers like Poetry, Hatch, uv, and Pipenv to guarantee reproducible builds across environments.

```
# Create and activate virtual environment python -m venv .venv source
.venv/bin/activate
```